C E R E M E M O R Y

P R O T O C O L S P E C I F I C A T I O N

CMP (Cerememory Protocol) v1.0

The Standard Interface for LLM-Agnostic Memory

Version 1.0 | March 2026

Author: Masato Okuwaki | CORe Inc.

Companion to: Cerememory Whitepaper v0.1

1. Introduction

1.1 Purpose

The Cerememory Protocol (CMP) defines a standard interface for reading from and writing to a Cerememory instance. It enables any LLM, application, or agent to interact with a brain-inspired, living memory system without knowledge of the underlying storage implementation.

CMP is designed to be transport-agnostic. It specifies message formats and semantics, not wire protocols. Implementations may use HTTP/REST, gRPC, WebSocket, Unix domain sockets, or in-process function calls.

1.2 Scope

This specification covers:

- Core data types and structures
- The four operation categories: Encode, Recall, Lifecycle, and Introspect
- Error handling semantics
- Versioning and extension mechanisms

This specification does not cover:

- Internal storage engine implementation details
- LLM adapter implementation (model-specific serialization/parsing)
- Encryption and authentication mechanisms (deferred to a security companion spec)

1.3 Conformance

A conformant CMP implementation MUST support all operations marked as REQUIRED in this specification. Operations marked OPTIONAL may be omitted but, if implemented, MUST conform to the specified behavior. The key words MUST, MUST NOT, SHOULD, SHOULD NOT, and MAY follow RFC 2119 semantics.

1.4 Notational Conventions

Data structures in this specification are described using TypeScript-style type notation for readability. This does not imply a required implementation language. All timestamps are ISO 8601 UTC. All identifiers are UUIDv7 unless otherwise specified.

2. Core Data Types

2.1 MemoryRecord

The fundamental unit of storage in Cerememory. Every piece of stored information is represented as a MemoryRecord, regardless of which store it resides in.

```
type MemoryRecord = {
  id: UUIDv7;
  store: StoreType;
  created_at: ISO8601;
  updated_at: ISO8601;
  last_accessed_at: ISO8601;
  access_count: uint32;
  content: MemoryContent;
  fidelity: FidelityState;
  emotion: EmotionVector;
  associations: Association[];
  metadata: Record<string, any>;
  version: uint32;
};
```

2.2 StoreType

Identifies the target memory store. Each store has distinct storage semantics and retrieval behavior.

```
type StoreType = "episodic" | "semantic" | "procedural"
               | "emotional" | "working";
```

Store	Content Model	Retrieval Model
`episodic`	Temporal event logs with spatial context, multimodal payloads	Temporal range, spatial proximity, cue-based pattern completion
`semantic`	Graph nodes (entities/concepts) and typed edges	Graph traversal, spreading activation, semantic similarity
`procedural`	Statistical profiles, behavioral patterns, distributional representations	Pattern matching, profile scoring
`emotional`	Cross-cutting emotion vectors attached to records in other stores	Emotion-filtered queries, affective similarity
`working`	Volatile, limited-capacity active context	Direct access by recency, capacity-bounded eviction

2.3 MemoryContent

The payload of a memory record. Content is multimodal by design. A single record may contain multiple content blocks of different modalities.

```
type MemoryContent = {
  blocks: ContentBlock[];
  summary: string | null;
};

type ContentBlock = {
  modality: Modality;
  format: string;       // MIME type or format identifier
  data: bytes | string;  // Raw data or text
  embedding: float[] | null;  // Optional pre-computed embedding
};

type Modality = "text" | "image" | "audio" | "video"
              | "structured" | "spatial" | "temporal"
              | "interoceptive";
```

2.4 FidelityState

Represents the current decay state of a memory record. This is the mechanism by which memories fade and noise accumulates over time.

```
type FidelityState = {
  score: float;            // 0.0 (fully decayed) to 1.0 (pristine)
  noise_level: float;      // 0.0 (no noise) to 1.0 (fully noisy)
  decay_rate: float;       // Per-second decay coefficient
  emotional_anchor: float; // Emotional modulation (higher = slower decay)
  reinforcement_count: uint32;  // Times reactivated
  last_decay_tick: ISO8601;
};
```

2.5 EmotionVector

A multi-dimensional affective representation. Based on Plutchik's wheel with continuous intensity values.

```
type EmotionVector = {
  joy: float;           // 0.0 to 1.0
```

```
  trust: float;
  fear: float;
  surprise: float;
  sadness: float;
  disgust: float;
  anger: float;
  anticipation: float;
  intensity: float;     // Overall emotional intensity
  valence: float;       // -1.0 (negative) to 1.0 (positive)
};
```

2.6 Association

A weighted, typed, bidirectional link between two memory records. Associations are the substrate for spreading activation and cross-modal recall.

```
type Association = {
  target_id: UUIDv7;
  type: AssociationType;
  weight: float;          // 0.0 to 1.0
  created_at: ISO8601;
  last_co_activation: ISO8601;
};

type AssociationType = "temporal"    // Co-occurred in time
                     | "spatial"     // Co-occurred in space
                     | "semantic"    // Conceptually related
                     | "emotional"   // Similar affective state
                     | "causal"      // Cause-effect relationship
                     | "sequential"  // Ordered sequence
                     | "cross_modal" // Links across modalities
                     | "user_defined";
```

3. Encode Operations

Encode operations write new memories into Cerememory. The caller provides content and metadata; Cerememory handles store routing, fidelity initialization, association inference, and index creation.

3.1 encode.store [REQUIRED]

Store a new memory record.

Request

```
type EncodeStoreRequest = {
  content: MemoryContent;
  store: StoreType | null;        // null = auto-route
  emotion: EmotionVector | null;   // null = auto-detect
  context: EncodeContext | null;
  associations: ManualAssociation[] | null;
};

type EncodeContext = {
  source: string;          // e.g. 'conversation', 'document', 'observation'
  session_id: string | null;
  spatial: SpatialContext | null;
  temporal: TemporalContext | null;
};
```

Response

```
type EncodeStoreResponse = {
  record_id: UUIDv7;
  store: StoreType;             // Actual store used (relevant if auto-routed)
  initial_fidelity: float;
  associations_created: uint32;  // Auto-inferred associations
};
```

If store is null, Cerememory MUST analyze the content and route it to the most appropriate store. Mixed-type content MAY result in records being created in multiple stores with cross-references.

3.2 encode.batch [REQUIRED]

Store multiple memory records in a single transaction. Useful for encoding an entire conversation or document.

Request

```
type EncodeBatchRequest = {
  records: EncodeStoreRequest[];
  infer_associations: boolean;  // Auto-detect associations between batch items
};
```

Response

```
type EncodeBatchResponse = {
  results: EncodeStoreResponse[];
  associations_inferred: uint32;
};
```

3.3 encode.update [REQUIRED]

Update an existing memory record. This is an explicit write, distinct from the implicit reconsolidation that occurs on recall.

```
type EncodeUpdateRequest = {
  record_id: UUIDv7;
  content: Partial<MemoryContent> | null;
  emotion: Partial<EmotionVector> | null;
  metadata: Record<string, any> | null;
};
```

4. Recall Operations

Recall operations retrieve memories from Cerememory. Recall is not a passive read; it is an active process that may trigger reconsolidation, spreading activation, and fidelity updates.

4.1 recall.query [REQUIRED]

The primary recall operation. Accepts a multimodal cue and returns matching memories ranked by relevance, recency, and fidelity.

Request

```
type RecallQueryRequest = {
  cue: RecallCue;
  stores: StoreType[] | null;       // null = search all stores
  limit: uint32;                     // Max results (default: 10)
  min_fidelity: float | null;        // Filter threshold (default: 0.0)
  include_decayed: boolean;           // Include low-fidelity memories
  reconsolidate: boolean;             // Trigger reconsolidation (default: true)
  activation_depth: uint32;           // Spreading activation hops (default: 2)
  recall_mode: RecallMode;
};

type RecallCue = {
  text: string | null;
  image: bytes | null;
  audio: bytes | null;
  emotion: EmotionVector | null;
  temporal: TemporalRange | null;
  spatial: SpatialRange | null;
  semantic: SemanticFilter | null;
};

  type RecallMode = "human"     // Apply fidelity/noise (default)
                  | "perfect";  // Return original data, no noise
```

Response

```
type RecallQueryResponse = {
  memories: RecalledMemory[];
  activation_trace: ActivationTrace | null;  // If requested
  total_candidates: uint32;
};
```

```
type RecalledMemory = {
  record: MemoryRecord;
  relevance_score: float;
  activation_path: UUIDv7[] | null;  // How this memory was reached
  rendered_content: MemoryContent;    // After fidelity/noise applied
};
```

When recall_mode is "human", the rendered_content field contains the memory as it would be experienced: with fidelity-weighted noise applied. Low-fidelity memories return blurred, degraded content. When recall_mode is "perfect", rendered_content returns the original stored data with no degradation.

When reconsolidate is true, the recall operation MUST update the accessed record: incrementing access_count, updating last_accessed_at, applying a small fidelity boost (reinforcement), and optionally incorporating the current query context into the record (reconsolidation). Implementations SHOULD make the reconsolidation effect configurable.

4.2 recall.associate [REQUIRED]

Given a specific memory record, retrieve its associated memories via spreading activation.

```
type RecallAssociateRequest = {
  record_id: UUIDv7;
  association_types: AssociationType[] | null; // null = all types
  depth: uint32;             // Hops to traverse (default: 2)
  min_weight: float;         // Minimum association weight (default: 0.1)
  limit: uint32;
};
```

4.3 recall.timeline [OPTIONAL]

Retrieve episodic memories in temporal order, producing a narrative timeline.

```
type RecallTimelineRequest = {
  range: TemporalRange;
  granularity: TimeGranularity;  // 'minute' | 'hour' | 'day' | 'week' | 'month'
  min_fidelity: float | null;
  emotion_filter: Partial<EmotionVector> | null;
};
```

4.4 recall.graph [OPTIONAL]

Retrieve a subgraph from the semantic store. Returns nodes and edges for visualization or traversal.

```
type RecallGraphRequest = {
  center_id: UUIDv7 | null;      // null = full graph (paginated)
  depth: uint32;
  edge_types: string[] | null;
  limit_nodes: uint32;
};
```

5. Lifecycle Operations

Lifecycle operations manage the temporal dynamics and evolution of the memory system.

5.1 lifecycle.consolidate [REQUIRED]

Trigger a consolidation cycle. This migrates mature episodic memories to the semantic store, abstracting and compressing them. Analogous to the memory consolidation that occurs during sleep.

```
type ConsolidateRequest = {
  strategy: "full" | "incremental" | "selective";
  min_age_hours: uint32;        // Only consolidate records older than this
  min_access_count: uint32;     // Only consolidate sufficiently accessed records
  dry_run: boolean;             // Preview without executing
};

type ConsolidateResponse = {
  records_processed: uint32;
  records_migrated: uint32;
  records_compressed: uint32;
  records_pruned: uint32;
  semantic_nodes_created: uint32;
};
```

5.2 lifecycle.decay_tick [REQUIRED]

Advance the decay engine by one tick. The tick interval is implementation-defined. This operation updates fidelity scores and noise levels for all active records.

```
type DecayTickRequest = {
  tick_duration_seconds: uint32 | null;  // null = use configured default
};

type DecayTickResponse = {
  records_updated: uint32;
  records_below_threshold: uint32;
  records_pruned: uint32;
};
```

5.3 lifecycle.set_mode [REQUIRED]

Switch the decay and recall behavior between human-realistic and perfect-recall modes.

```
type SetModeRequest = {
  mode: "human" | "perfect";
  scope: StoreType[] | null;     // null = all stores
};
```

5.4 lifecycle.forget [REQUIRED]

Explicitly delete memory records. Unlike natural decay, this is permanent and irreversible.

```
type ForgetRequest = {
  record_ids: UUIDv7[] | null;         // Specific records
  store: StoreType | null;              // All records in a store
  temporal_range: TemporalRange | null; // Time-based deletion
  cascade: boolean;    // Also remove associated records (default: false)
  confirm: boolean;    // Safety flag, MUST be true to execute
};
```

5.5 lifecycle.export [REQUIRED]

Export the complete memory state as a portable archive. The archive format is self-contained and includes all stores, associations, fidelity states, and evolution parameters.

```
type ExportRequest = {
  format: "cma";      // Cerememory Archive (required format)
  stores: StoreType[] | null;  // null = all
  encrypt: boolean;
  encryption_key: string | null;  // null = use instance default
};

type ExportResponse = {
  archive_id: string;
  size_bytes: uint64;
  record_count: uint32;
  checksum: string;
};
```

5.6 lifecycle.import [REQUIRED]

Import a Cerememory Archive into the current instance. Supports merge and replace strategies.

```
type ImportRequest = {
```

```
  archive_id: string;
  strategy: "merge" | "replace";
  conflict_resolution: "keep_existing" | "keep_imported" | "keep_newer";
  decryption_key: string | null;
};
```

6. Introspection Operations

Introspection operations allow users and applications to observe and understand the state of the memory system. Transparency is a core principle of Cerememory.

6.1 introspect.stats [REQUIRED]

Return aggregate statistics about the memory system.

```
type StatsResponse = {
  total_records: uint32;
  records_by_store: Record<StoreType, uint32>;
  total_associations: uint32;
  avg_fidelity: float;
  avg_fidelity_by_store: Record<StoreType, float>;
  oldest_record: ISO8601;
  newest_record: ISO8601;
  total_recall_count: uint64;
  evolution_metrics: EvolutionMetrics;
};
```

6.2 introspect.record [REQUIRED]

Return the full state of a specific memory record, including its fidelity history, association graph, and reconsolidation log.

```
type RecordIntrospectRequest = {
  record_id: UUIDv7;
  include_history: boolean;       // Fidelity change history
  include_associations: boolean;  // Full association graph
  include_versions: boolean;      // Reconsolidation history
};
```

6.3 introspect.decay_forecast [OPTIONAL]

Predict the fidelity state of a record or set of records at a future point in time, assuming no intervening access.

```
type DecayForecastRequest = {
  record_ids: UUIDv7[];
  forecast_at: ISO8601;     // Target future timestamp
};
```

```
type DecayForecastResponse = {
  forecasts: {
    record_id: UUIDv7;
    current_fidelity: float;
    forecasted_fidelity: float;
    estimated_threshold_date: ISO8601 | null;  // When it drops below recall
threshold
  }[];
};
```

6.4 introspect.evolution [OPTIONAL]

Return the current state of the evolution engine: which parameters have been tuned, what usage patterns have been detected, and what architectural adaptations have been made.

```
type EvolutionMetrics = {
  parameter_adjustments: {
    store: StoreType;
    parameter: string;
    original_value: float;
    current_value: float;
    reason: string;
  }[];
  detected_patterns: string[];
  schema_adaptations: string[];
};
```

7. Error Handling

All CMP operations return errors using a standardized error envelope.

```
type CMPError = {
  code: ErrorCode;
  message: string;
  details: Record<string, any> | null;
  retry_after: uint32 | null;  // Seconds, for rate-limited operations
};
```

Error Codes

Code	Category	Description
`RECORD_NOT_FOUND`	Client	The specified record ID does not exist
`STORE_INVALID`	Client	Invalid or unsupported store type
`CONTENT_TOO_LARGE`	Client	Content exceeds implementation size limit
`MODALITY_UNSUPPORTED`	Client	Content modality not supported by this implementation
`WORKING_MEMORY_FULL`	Capacity	Working memory cache at capacity; eviction required
`DECAY_ENGINE_BUSY`	System	Decay engine is processing; retry after interval
`CONSOLIDATION_IN_PROGRESS`	System	A consolidation cycle is running; conflicting write blocked
`EXPORT_FAILED`	System	Archive export failed (disk, encryption, etc.)
`IMPORT_CONFLICT`	Client	Import conflicts detected; resolution required
`FORGET_UNCONFIRMED`	Client	Forget operation requires confirm: true
`VERSION_MISMATCH`	Protocol	Client CMP version incompatible with server
`INTERNAL_ERROR`	System	Unrecoverable internal error

8. Versioning and Extensions

8.1 Protocol Versioning

CMP uses semantic versioning (MAJOR.MINOR). The MAJOR version increments on breaking changes. The MINOR version increments on backward-compatible additions.

All CMP messages MUST include a protocol version header:

```
type CMPHeader = {
  protocol: "cmp";
  version: "1.0";
  request_id: UUIDv7;
  timestamp: ISO8601;
};
```

Implementations MUST reject requests with an incompatible MAJOR version and SHOULD accept requests with a lower MINOR version by ignoring unknown fields.

8.2 Extension Mechanism

CMP supports extensions for adding new store types, content modalities, association types, and operations without modifying the core specification.

```
// Extensions are namespaced to avoid conflicts
// Convention: 'x-{vendor}-{name}'

// Example: custom store type
// store: "x-acme-spatial3d"

// Example: custom modality
// modality: "x-acme-haptic"

// Example: custom operation
// operation: "x-acme-dream_consolidate"
```

Extensions MUST NOT override the behavior of core operations. Implementations that do not recognize an extension MUST return MODALITY_UNSUPPORTED or an appropriate error, never silently ignore it.

9. Transport Bindings

CMP is transport-agnostic. This section defines reference bindings for common transport protocols.

9.1 HTTP/REST Binding

The HTTP binding maps CMP operations to REST endpoints:

Method	Path	Operation
POST	/v1/encode	encode.store
POST	/v1/encode/batch	encode.batch
PATCH	/v1/encode/{record_id}	encode.update
POST	/v1/recall/query	recall.query
POST	/v1/recall/associate/{id}	recall.associate
POST	/v1/recall/timeline	recall.timeline
POST	/v1/recall/graph	recall.graph
POST	/v1/lifecycle/consolidate	lifecycle.consolidate
POST	/v1/lifecycle/decay-tick	lifecycle.decay_tick
PUT	/v1/lifecycle/mode	lifecycle.set_mode
DELETE	/v1/lifecycle/forget	lifecycle.forget
POST	/v1/lifecycle/export	lifecycle.export
POST	/v1/lifecycle/import	lifecycle.import
GET	/v1/introspect/stats	introspect.stats
GET	/v1/introspect/record/{id}	introspect.record
POST	/v1/introspect/decay-forecast	introspect.decay_forecast
GET	/v1/introspect/evolution	introspect.evolution

9.2 gRPC Binding

A gRPC binding will be specified using Protocol Buffers in a companion .proto file. The service definition mirrors the operation categories: EncodeService, RecallService, LifecycleService, IntrospectService.

9.3 In-Process Binding

For embedded deployments (e.g., a Cerememory instance running inside a desktop application), a direct function-call interface is provided. The function signatures mirror the CMP operations directly, bypassing serialization overhead. Language-specific bindings will be published as SDKs.

10. Security Considerations

Memory data is among the most sensitive classes of personal information. While a full security specification is deferred to a companion document, the following requirements apply to all CMP implementations:

- **Encryption at Rest [REQUIRED]:** All memory stores MUST be encrypted at rest using AES-256 or equivalent. The encryption key MUST be held by the user, not by any service provider.
- **Encryption in Transit [REQUIRED]:** All CMP messages transmitted over a network MUST use TLS 1.3 or later.
- **Authentication [REQUIRED]:** All CMP operations MUST be authenticated. The authentication mechanism is transport-specific (e.g., Bearer tokens for HTTP, mTLS for gRPC).
- **Audit Logging [SHOULD]:** Implementations SHOULD maintain an append-only audit log of all encode, forget, export, and import operations.
- **Forget Verification [REQUIRED]:** The lifecycle.forget operation MUST cryptographically erase data, not merely mark it as deleted. After a forget operation, it MUST be computationally infeasible to recover the deleted records.

11. Conformance Checklist

A conformant CMP v1.0 implementation MUST support:

Operation	Status	Notes
`encode.store`	**REQUIRED**	
`encode.batch`	**REQUIRED**	
`encode.update`	**REQUIRED**	
`recall.query`	**REQUIRED**	
`recall.associate`	**REQUIRED**	
`recall.timeline`	OPTIONAL	
`recall.graph`	OPTIONAL	
`lifecycle.consolidate`	**REQUIRED**	
`lifecycle.decay_tick`	**REQUIRED**	
`lifecycle.set_mode`	**REQUIRED**	
`lifecycle.forget`	**REQUIRED**	
`lifecycle.export`	**REQUIRED**	
`lifecycle.import`	**REQUIRED**	
`introspect.stats`	**REQUIRED**	
`introspect.record`	**REQUIRED**	
`introspect.decay_forecast`	OPTIONAL	
`introspect.evolution`	OPTIONAL	

12. Future Work

The following areas are identified for future specification versions:

- **CMP v1.1:** Multi-agent shared memory operations. Protocols for multiple agents accessing and contributing to a shared memory pool with conflict resolution and access control.
- **CMP v1.2:** Streaming recall. Real-time subscription to memory events (new encodings, association changes, fidelity threshold crossings) via WebSocket or Server-Sent Events.
- **CMP v2.0:** Evolution operations. Expose the architecture-level evolution engine through the protocol, enabling external orchestration of schema adaptations.
- **Security Companion Spec:** Detailed authentication, authorization, encryption, and audit specifications for Cerememory deployments.

End of CMP Specification v1.0

Cerememory is open source. Contributions welcome.

github.com/co-r-e/cerememory